AI RESPONSE VALIDATION SYSTEM

Project Documentation

Internship Project

Purpose: describe the project objective, architecture, modules, workflow, technology stack, testing approach, deployment considerations, and limitations. Verify implementation details against the latest source code before submission.

1. Project Overview

The AI Response Validation System is a Python-based application designed to assess AI-generated answers. A user provides a question, an AI response, and optionally a reference answer or reference document. The system evaluates the response across multiple dimensions and produces scores and an overall verdict. It supports individual evaluation and CSV-based batch evaluation, with results available through dashboard and history views.

2. Problem Statement

AI-generated responses may be incomplete, irrelevant, inaccurate, or contain claims unsupported by available evidence. Reviewing many responses manually can be time-consuming. This project provides a structured workflow for assessing responses and reviewing the resulting scores and verdicts.

3. Objectives

- Evaluate an AI response against a question and optional reference material.
- Assess relevance, accuracy, hallucination risk, and completeness using separate evaluation agents.
- Combine evaluation outputs into an overall verdict.
- Support single-response and CSV batch evaluation.
- Store evaluation records and display them in dashboard/history views.
- Allow users to configure and compare supported language-model providers.

4. Scope

The current application includes a Streamlit interface, Python evaluation logic, agent modules, a retrieval/knowledge-base component, SQLite storage, and batch evaluation. It is an evaluation aid; its scores are model-assisted assessments, not a guarantee of factual correctness.

5. Technology Stack

Technology	Purpose
Python	Core application logic
Streamlit	User interface, dashboard, and evaluation screens
SQLite	Local storage of evaluation records
FAISS	Vector similarity search in the retrieval component
Sentence Transformers	Text embeddings for retrieval
Groq API, Gemini API, Ollama	Language-model provider options in the project
Pandas / CSV	Batch input and tabular result handling
PDF generation library	PDF report export, if enabled in the current build

Confirm exact package names and versions in requirements.txt. Provider availability depends on configuration, credentials, network access, and runtime environment.

6. System Architecture

1. Input layer collects the question, AI response, and optional reference answer/document.
2. Streamlit interface presents single evaluation, batch evaluation, history, and dashboard features.
3. Evaluation orchestrator coordinates the workflow and gathers outputs from agents.
4. Evaluation agents assess relevance, accuracy, hallucination, and completeness.
5. Knowledge retrieval can find relevant reference chunks from the indexed corpus when configured.
6. Verdict agent combines available evaluation outputs into a final result/status.
7. SQLite persistence stores evaluation information.
8. Output layer displays scores, verdicts, history, dashboard summaries, and supported export features.

7. Main Modules

- **app.py —** Main Streamlit application and user interface.
- **orchestrator.py —** Coordinates evaluation steps and calls the relevant agents.
- **database.py —** Creates and interacts with the SQLite database.
- **agents/relevance_agent.py —** Assesses how well the response addresses the question.
- **agents/accuracy_agent.py —** Assesses response accuracy using the inputs and evidence available.
- **agents/hallucination_agent.py —** Assesses potentially unsupported or fabricated claims.
- **agents/completeness_agent.py —** Assesses coverage of important parts of the question.
- **agents/verdict_agent.py —** Produces a combined verdict from evaluation results.
- **kb/ —** Knowledge-base loading, preprocessing, chunking, embeddings, indexing, and retrieval.
- **batch_evaluator.py —** Supports processing multiple cases from batch input.

8. Functional Workflow

9. The user enters a question and AI response and may provide a reference answer or document.
10. The application validates the input and prepares the evaluation request.
11. Where configured, the retriever finds relevant reference material.
12. The orchestrator invokes evaluation agents and collects their outputs.
13. The verdict module combines the available results into an overall assessment.
14. The interface displays scores, status, and result details.
15. Results can be saved and reviewed in history/dashboard views.
16. For batch evaluation, the app reads CSV rows and processes cases according to its validation and error-handling logic.

9. Inputs and Outputs

Inputs

- Question to be answered.
- AI-generated response to evaluate.
- Optional reference answer.
- Optional reference document.
- CSV containing multiple cases with the column names expected by the application.

Outputs

- Scores for the evaluation dimensions.
- Overall score and verdict according to configured project logic.
- Evaluation history and dashboard summaries.
- Batch results and row-level error information where implemented.
- PDF export/report functionality where enabled.

10. Data Storage

The application uses SQLite through database.py to store evaluation records and related batch information. A local database file is local application data. For deployment, confirm whether data must persist across restarts and use durable database storage if required.

11. Testing and Validation

The project includes test files for selected agent and verdict behavior. Run tests and record actual results before submission; do not mark tests as passed unless they were executed successfully.

- Valid single-evaluation input.
- Missing or empty AI response.
- Input with a reference answer.
- Input with a reference document.
- CSV batch with multiple valid rows.
- CSV batch containing an invalid row.
- Each configured provider with valid credentials.
- Database history and dashboard updates.
- PDF export, if available.

12. Deployment Considerations

- Commit the source code to the project GitHub repository.
- Include a curated requirements.txt matching the imports and tested environment.
- Never commit .env files, API keys, or secrets; configure hosted secrets through the deployment platform.
- Streamlit Cloud cannot automatically access Ollama running only on the developer's computer.
- Include required knowledge-base/index files and check repository size limits.
- SQLite on an ephemeral hosted filesystem may not provide durable storage across restarts or redeployments.
- Test all essential workflows after deployment.

13. Limitations

- LLM-based evaluation can vary and may itself make mistakes.
- Scores depend on input quality, reference material, prompts, model, and evaluation rules.
- Retrieval can miss relevant evidence or return irrelevant chunks.
- Provider rate limits, network failures, credentials, or model availability can interrupt evaluation.
- Local Ollama access does not automatically transfer to a hosted cloud environment.
- The current database setup may need changes for durable multi-user production use.

14. Future Enhancements

- Add more benchmark datasets and a repeatable evaluation protocol.
- Improve evidence citations and claim-level explanations.
- Add retries, rate-limit handling, and clearer provider errors.
- Use durable database storage for multi-user deployment.
- Expand automated tests for batch processing, retrieval, database operations, and provider failures.
- Compare system scores with human-reviewed examples and report measured results.

15. Conclusion

The AI Response Validation System provides a modular workflow for reviewing AI-generated answers across multiple dimensions, producing an overall verdict, optionally retrieving reference material, and supporting single and batch evaluation. Verify the final implementation, run tests, record actual outcomes, and confirm deployment configuration before submission.